

```
1  #include <cs50.h>
2  #include <stdio.h>
3  #include <stdlib.h>
4
5  typedef struct node
6  {
7      int number;
8      struct node *next;
9  } node;
10
11 int main(void)
12 {
13     // Head of linked list
14     node *list = NULL;
15
16     // Iteratively create nodes
17     for (int i = 0; i < 3; i++)
18     {
19         int x = get_int("Number: ");
20
21         node *n = malloc(sizeof(node));
22         n->number = x;
23         n->next = list;
24
25         list = n;
26     }
27
28     // Traverse through linked list and print each number
29     for (node *ptr = list; ptr != NULL; ptr = ptr->next)
30     {
31         printf("%i\n", ptr->number);
32     }
33
34     // Free the linked list
35     node *ptr = list;
36     while (ptr != NULL)
37     {
38         node *tmp = ptr;
39         ptr = ptr->next;
40         free(tmp);
41     }
42 }
```

```
1  #include <cs50.h>
2  #include <ctype.h>
3  #include <stdio.h>
4  #include <string.h>
5
6  int hash(char *word);
7
8  int main(void)
9  {
10     char *word = get_string("Word: ");
11     printf("Hash value: %i\n", hash(word));
12 }
13
14 // Hash the word based on the first two letters of the word
15 int hash(char *word)
16 {
17     // Word is less than 2 letters
18     if (word == NULL || strlen(word) < 2)
19     {
20         return -2;
21     }
22
23     char c = word[0];
24     char c1 = word[1];
25
26     if (isalpha(c) && isalpha(c1))
27     {
28         c = toupper(c);
29         c1 = toupper(c1);
30         return (c - 'A') * 100 + c1 - 'A';
31     }
32     return -1;
33 }
```